DATA220 Section23

Math Method for DA

Pair Programming

Lab-2

Group 7

Aiswarya Raghavadesikan (014574599)
&
Aryama Ray (017401124)

TABLE OF CONTENTS

1. Introduction:

Data Analysis is the most crucial segment of data validation and exploration. This project will focus mostly on three different kinds of datasets segmented into three different parts. The project focuses on three different sectors/domains such as Entertainment, Real estates and Healthcare as three parts respectively.

2. Project Scope:

The scope of the project involves addressing the questions mentioned in the Lab2 report. Data cleaning, Data Visualizations, Statistical Modelings were involved to demonstrate the data analyst's perspective with respect to handling the problem dataset.

3. Dataset Explorations

3.1. Part1: MovieLens Dataset

3.1.1. Exploration of the dataset:

Loading of Datasets

Handling a new format of datafile **.dat** on python gave a great experience to learn and convert the file to an appropriate file format, here it is converted to .csv file.
.dat files are typically an unstructured data file or the text file which needs to be structured to proceed further.

First look:

	0
0	<!DOCTYPE HTML>
1	<html>
2	<head>
3	<meta charset="utf-8">
4	<title>Jupyter Notebook</title>

Procedure:

- File opened to read with appropriate encoding (latin-1)

- Formatted the files movies.dat, ratings.dat and users.dat
- Splitted the file based on the appropriate columns name as specified in the README file.
- Finally converted the .dat file to .csv file (after applying the above formats)

Movies dataset

```
1 data_movies.head()
```

	MovieID	Title	Genres
0	1	Toy Story (1995)	['Animation', "Children's", 'Comedy']
1	2	Jumanji (1995)	['Adventure', "Children's", 'Fantasy']
2	3	Grumpier Old Men (1995)	['Comedy', 'Romance']
3	4	Waiting to Exhale (1995)	['Comedy', 'Drama']
4	5	Father of the Bride Part II (1995)	['Comedy']

Data Exploration: For "Movies" dataset

Categorical data:

- **Title :** The string datatype describing the movie title.
- **Genres :** The string datatype describing the movie title.

Discrete data:

- **MovieID :** Discrete data giving the information about the unique entries of Movie ID.

Ratings dataset

```
1 data_ratings.head()
```

	UserID	MovieID	Rating	Timestamp
0	1	1193	5.0	2000-12-31 22:12:40
1	1	661	3.0	2000-12-31 22:35:09
2	1	914	3.0	2000-12-31 22:32:48
3	1	3408	4.0	2000-12-31 22:04:35
4	1	2355	5.0	2001-01-06 23:38:11

Data Exploration: For "Ratings" dataset

Discrete data:

- **UserID :** Discrete data (integer datatype) giving the information about the unique entries of UserID rating one or more movies.
- **MovieID:** Discrete data (integer datatype) giving the information about the unique entries of MovieID receiving ratings from one or more users.

Continuous data:

- **Ratings :** Continuous data (float datatype) giving information about the movie's ratings.

Continuous data:

- **Timestamp:** Timestamp with date refers to categorical, ordinal, continuous or temporal data based on the context. Here we can say it is a continuous data role as it receives ratings on a particular date with timestamp details**.**

Users dataset

```
1 data_users.head()
```

	UserID	Gender	Age	Occupation	ZipCode
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

Data Exploration: For "Users" dataset

Discrete data:

- **UserID :** Discrete data (integer datatype) giving the information about the unique entries of UserID.
- **Age:** Discrete data (integer datatype) showing the information about the ages of the users.
- **Occupation:** Discrete data (integer datatype) showing the information about the Occupation number of the users.
- **Zip Code:** Discrete data (integer datatype) showing the information about the ZipCode of the users who provides the ratings to the movies.

Categorical:

- **Gender : Categorical data (String data type) having Male "M" or Female "F" based on the User details.**

3.1.2. Singular Value Decomposition (SVD)

Singular Value Decomposition or shortly termed as SVD is a powerful tool in data science, machine learning models for understanding, analyzing, and simplifying complex datasets. Any given matrix (mxn) can be decomposed into three other matrices, providing a way to represent the original matrix in a simplified form. Hence SVD becomes useful in analyzing and reducing the dimensionality of data while retaining important features.

Given a matrix A, SVD decomposes it following three matrices:

$A = U\Sigma V^T$

Where,
U is an orthogonal matrix representing the left singular vectors of A.
Σ is a diagonal normalized eigenvector matrix containing the singular values of A.
V^T is the transpose of an orthogonal matrix representing the right singular vectors of A.

Formula:
Given A matrix which is of (m * n) shape.

A = U + Σ + V^T

(m * n = m * m + m * n + n * n)

Some of the key applications of SVD in data science are Image compression, Dimensionality reduction, Matrix approximation.

Content-based vs Collaborative recommendation.

The choice between content-based or collaborative or perhaps an hybrid approach depends on the characteristics of the data, the quality of available features, and the specific goals of the recommendation system.

Content-based recommendation relies on the characteristics or features of items (movies in the given dataset) and the preferences expressed by the user. The idea is to recommend items similar to those the user has liked in the past based on the content features of the items.

Collaborative recommendation is based on the idea that users who agreed in the past tend to

agree in the future. It doesn't rely on item features but rather on the historical preferences and behaviors of users. Collaborative filtering can be user-based (recommend items to a user based on the preferences of users similar to them) or item-based (recommend items to a user based on the preferences of other users who liked similar items).

Implementation of SVD:

Creating movies users matrix (m*u) from the given dataset

```
1 mxu_matrix.head()
```

UserID	1	2	3	4	5	6	7	8	9	10	...	6031	6032	6033	6034	6035	6036	6037	6038	6039	6040
MovieID																					
1	5.0	0.0	0.0	0.0	0.0	4.0	0.0	4.0	5.0	5.0	...	0.0	4.0	0.0	0.0	4.0	0.0	0.0	0.0	0.0	3.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	5.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	3.0	0.0	0.0	...	0.0	0.0	0.0	0.0	2.0	2.0	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0

5 rows × 6040 columns

Normalization of the m*u matrix

```
1 mxu_normalize.head()
```

UserID	1	2	3	4	5	6	7	8	9	10	...	6031	6032	6033	
MovieID															
1	3.574007	-1.425993	-1.425993	-1.425993	-1.425993	2.574007	-1.425993	2.574007	3.574007	3.574007	...	-1.425993	2.574007	-1.425993	-1.42
2	-0.371523	-0.371523	-0.371523	-0.371523	-0.371523	-0.371523	-0.371523	-0.371523	-0.371523	4.628477	...	-0.371523	-0.371523	-0.371523	-0.37
3	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	-0.238742	...	-0.238742	-0.238742	-0.238742	-0.23
4	-0.076821	-0.076821	-0.076821	-0.076821	-0.076821	-0.076821	-0.076821	2.923179	-0.076821	-0.076821	...	-0.076821	-0.076821	-0.076821	-0.07
5	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	-0.147351	...	-0.147351	-0.147351	-0.147351	-0.14

5 rows × 6040 columns

- In order to perform SVD, we need to create a movies and users matrix as rows and columns respectively based on the ratings matrix.
- The Ratings values are pivoted and to assign the values to zero to enable the matrix to be included with the normalized ratings values.

Decomposed Matrix after SVD:

The results for U, S and V after decomposing the original matrix using SVD is given below:

```
# Package Import
from scipy.linalg import svd
```

```
U, S, V = svd(mxu_normalize, full_matrices=False)

```

```
sigma_matrix = np.diag(S)
```

```
print(f"U : {U.shape}, sigma: {sigma_matrix.shape}, V: {V.shape}")
```

```
U : (3706, 3706), sigma: (3706, 3706), V: (3706, 6040)
```

- Using scipy linalg package svd is performed.
- The matrix decomposition is possible when we normalize the matrix and perform a Singular Value decomposition to it to get a constructed resultant matrix.
- Here we have, U : (3706, 3706), sigma: (3706, 3706), V: (6040, 6040) on setting the full matrices to True.
- However, the matrix we could get here after SVD is a rectangular matrix on the V matrix U : (3706, 3706), sigma: (3706, 3706), V: (3706, 6040) as per the normalization process carried out.
- SVD Matrix shape is 3706 * 6040

Selection of top 50 components from S:

```
Sigma_k_50.shape
```

```
(50, 50)
```

```
#Printing the new U, Sigma and V for K = 50 components
(U_k.shape, Sigma_k_50.shape, V_k.shape)
```

```
((3706, 50), (50, 50), (50, 6040))
```

```
1 Sigma_k_50
```

```
array([[1298.7371535 ,    0.        ,    0.        , ...,    0.        ,
           0.        ,    0.        ],
       [   0.        ,  671.34354097,    0.        , ...,    0.        ,
           0.        ,    0.        ],
       [   0.        ,    0.        ,  574.54798415, ...,    0.        ,
           0.        ,    0.        ],
       ...,
       [   0.        ,    0.        ,    0.        , ...,  149.32202449,
           0.        ,    0.        ],
       [   0.        ,    0.        ,    0.        , ...,    0.        ,
         147.50259488,    0.        ],
       [   0.        ,    0.        ,    0.        , ...,    0.        ,
           0.        ,  146.84737883]])
```

- Using csr_matrix, it allows efficient arithmetic operations on sparse matrices, making it suitable for mathematical computations involving sparse data.
- Here it is performed to make it more memory-efficient compared to dense representations for matrices with a significant number of zero entries.
- After selecting the top 50 components we can observe the size of the matrices changed as per the requirement:
    - U_k = (3706, 50)
    - Sigma_k_50 = (50, 50)
    - V_k = (50, 6040)
- Note: k denotes top 50 components

Obtaining top 50 eigenvectors using eigenvalues:

The top 50 eigenvectors for the corresponding eigenvalues are:

```
1 eigen_vectors_50
```

```
array([[-7.13393053e-03+0.j,  1.64099327e-03+0.j, -2.14622406e-03+0.j,
        ...,  5.93713869e-03+0.j,  5.95650709e-03+0.j,
       -1.92305260e-03+0.j],
       [-6.40383513e-04+0.j, -2.70126226e-03+0.j,  2.00478360e-04+0.j,
        ..., -1.02650386e-05+0.j, -6.98621978e-03+0.j,
         1.28333884e-02+0.j],
       [-6.72473390e-03+0.j, -3.34737240e-03+0.j,  3.95617989e-03+0.j,
        ..., -2.71745194e-03+0.j, -6.17870453e-03+0.j,
       -1.71058911e-02+0.j],
       ...,
       [-1.13666709e-02+0.j,  1.80896437e-03+0.j,  5.62198713e-04+0.j,
        ...,  3.86190451e-03+0.j, -1.21471781e-02+0.j,
         2.75605839e-03+0.j],
       [-3.49381899e-03+0.j,  1.87620989e-02+0.j,  1.08962191e-02+0.j,
        ...,  1.50272389e-03+0.j, -5.32028005e-03+0.j,
       -8.38519634e-04+0.j],
       [ 1.32856412e-02+0.j,  4.08015550e-02+0.j,  3.63311909e-03+0.j,
        ..., -1.90445430e-02+0.j,  9.47302822e-04+0.j,
         1.48134117e-02+0.j]])
```

```
1 # For top 50 eigenvectors
2 eigen_vectors_50 = eigen_vectors[:, eigen_indices[:50]]
3
4 eigen_vectors_50.shape
```

```
(6040, 50)
```

- The purpose of eigenvalues and eigenvectors in machine learning models is to help understand how the data performs in a coordinate system.
- Process of rotating axis, stretching based on scalar eigen values and rotating to have a different view of the data.
- Various use cases of EigenValue and eigenVectors are Principal Component Analysis, Image compressions and reconstructions, Google PageRank algorithm.

Cosine similarity: Finding 10 closest movies using the 50 components from SVD:
The 10 closest movies using the top 50 singular values after performing cosine similarity are:

```
12
13 # Finding the 10 closest movies
14 closest_10_movies_indices = np.argsort(scores_cosine_similarity)[::-1][1:11]
15
16 closest_10_moviestitles = movies_df.iloc[closest_10_movies_indices]['Title']
17
18 closest_10_moviestitles
```

```
2898           Bad Seed, The (1956)
33                      Babe (1995)
2162                 Rounders (1998)
2128                Firelight (1997)
2191                   Wisdom (1986)
574     Hour of the Pig, The (1993)
2941                  Rosetta (1999)
517        Romeo Is Bleeding (1993)
2483     My Boyfriend's Back (1993)
1743                   Paulie (1998)
Name: Title, dtype: object
```

- The purpose of cosine similarity is to measure the cosine of the angle between two vectors, providing a metric for similarity in orientation regardless of vector magnitude.
- Displaying the MovieID and Title of the movie based on the movie_index that is specified along with all other movies in the dataset where the Movies with the highest similarity scores are fetched.
- Finally the result of top 10 closest movies are chosen based on their feature vectors denoted by the normalized_U_k matrix here.
- Use cases: Text mining, information retrieval, and recommendation systems to assess the similarity between documents or items represented as vectors.

Results of above SVD methods:

For k= 50 outcome:

- Helped to choose the most significant features or patterns in the data.
- Helped to condense the mxu_matrix to

Cosine similarity for closest top 10 movies:

- Based on the reduced dimensionality the cosine similarity is performed.
- Can infer that the movies with higher cosine similarity are likely to be enjoyed by similar groups of users.
- Hence we can say that the **top 10 movies** identified using cosine similarity are the movies that are most similar to a particular reference movie in terms of user ratings.
- From SVD, we obtained these recommendations as they are made by comparing the reduced-dimensional representations of movies.

Overall outcome:

- The results provide insights into which movies are most similar to each other based on user ratings, considering the reduced-dimensional representation obtained through SVD.

3.1.3. Conclusion:

SVD helped in simplifying the dimensionality of the given dataset in order to find the top 10 closest movies based on the users ratings for the condition k=50 (singular value). This will help in predicting the recommendations for the users to select the movies based on the users preferences.

3.2. Part2: House Prices Prediction Dataset

3.2.1. Exploration of the dataset.

```
1 house_price.dtypes
```

```
date              object
bedrooms           int64
bathrooms        float64
sqft_living        int64
sqft_lot           int64
floors           float64
waterfront         int64
view               int64
condition          int64
sqft_above         int64
sqft_basement      int64
yr_built           int64
yr_renovated       int64
SalesPrice       float64
dtype: object
```

- House Price dataset has **4600 records and 14 columns** such as: date, bedroom count, bathroom count, living area sqft details, lot sqft , number of floors, waterfront option, view, condition of the house, sqft above and basement sqft area, year when the house was built, renovation year etc.
- These attributes contribute towards determining the **price** of the house along with the variable - **SalesPrice** , which is the selling price of the house.
- So the **target variable** for these dataset is **SalesPrice** whereas other variables are different features.

Checking null values in the dataset:

```
1 house_price.isnull().sum()
```

```
date              0
bedrooms          2
bathrooms         2
sqft_living       0
sqft_lot          0
floors            0
waterfront        0
view              0
condition         0
sqft_above        0
sqft_basement     0
yr_built          0
yr_renovated      0
SalesPrice       49
dtype: int64
```

- Bedrooms and Bathrooms count are numeric categorical variables. So we will replace null values for these attributes with corresponding mode values.
- Sales Price for a house is continuous variable, hence null imputation will be done using median values.

Aggregation of zero values in each column:

```
date_zero_cnt : 0
bedrooms_zero_cnt : 2
bathrooms_zero_cnt : 2
sqft_living_zero_cnt : 0
sqft_lot_zero_cnt : 0
floors_zero_cnt : 0
waterfront_zero_cnt : 4567
view_zero_cnt : 4140
condition_zero_cnt : 0
sqft_above_zero_cnt : 0
sqft_basement_zero_cnt : 2745
yr_built_zero_cnt : 0
yr_renovated_zero_cnt : 2735
SalesPrice_zero_cnt : 49
```

Following attributes are having zero value fields:

- bedrooms_zero_cnt,
- bathrooms_zero_cnt,
- waterfront_zero_cnt,
- view_zero_cnt,
- sqft_basement_zero_cnt,
- yr_renovated_zero_cnt
- SalesPrice_zero_cnt

But, considering the house sales price prediction, a house might not have view or waterfront. So, the corresponding count might be zero. Also, if it is new house, then renovation year might become zero.A house may not have a basement as well which justifies the basement sqft value as zero. But Salesprice cannot be zero given the house is already sold.

We will replace the rest of the zero values with null values.

Summarizing the dataset:
The statistical summary for the continuous features are listed below:

```
1 house_price.describe()
```

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built
count	4598.000000	4598.000000	4600.000000	4.600000e+03	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000
mean	3.402349	2.161755	2139.346957	1.485252e+04	1.512065	0.007174	0.240652	3.451739	1827.265435	312.081522	1970.786304
std	0.906273	0.782654	963.206916	3.588444e+04	0.538288	0.084404	0.778405	0.677230	862.168977	464.137228	29.731848
min	1.000000	0.750000	370.000000	6.380000e+02	1.000000	0.000000	0.000000	1.000000	370.000000	0.000000	1900.000000
25%	3.000000	1.750000	1460.000000	5.000750e+03	1.000000	0.000000	0.000000	3.000000	1190.000000	0.000000	1951.000000
50%	3.000000	2.250000	1980.000000	7.683000e+03	1.500000	0.000000	0.000000	3.000000	1590.000000	0.000000	1976.000000
75%	4.000000	2.500000	2620.000000	1.100125e+04	2.000000	0.000000	0.000000	4.000000	2300.000000	610.000000	1997.000000
max	9.000000	8.000000	13540.000000	1.074218e+06	3.500000	1.000000	4.000000	5.000000	9410.000000	4820.000000	2014.000000

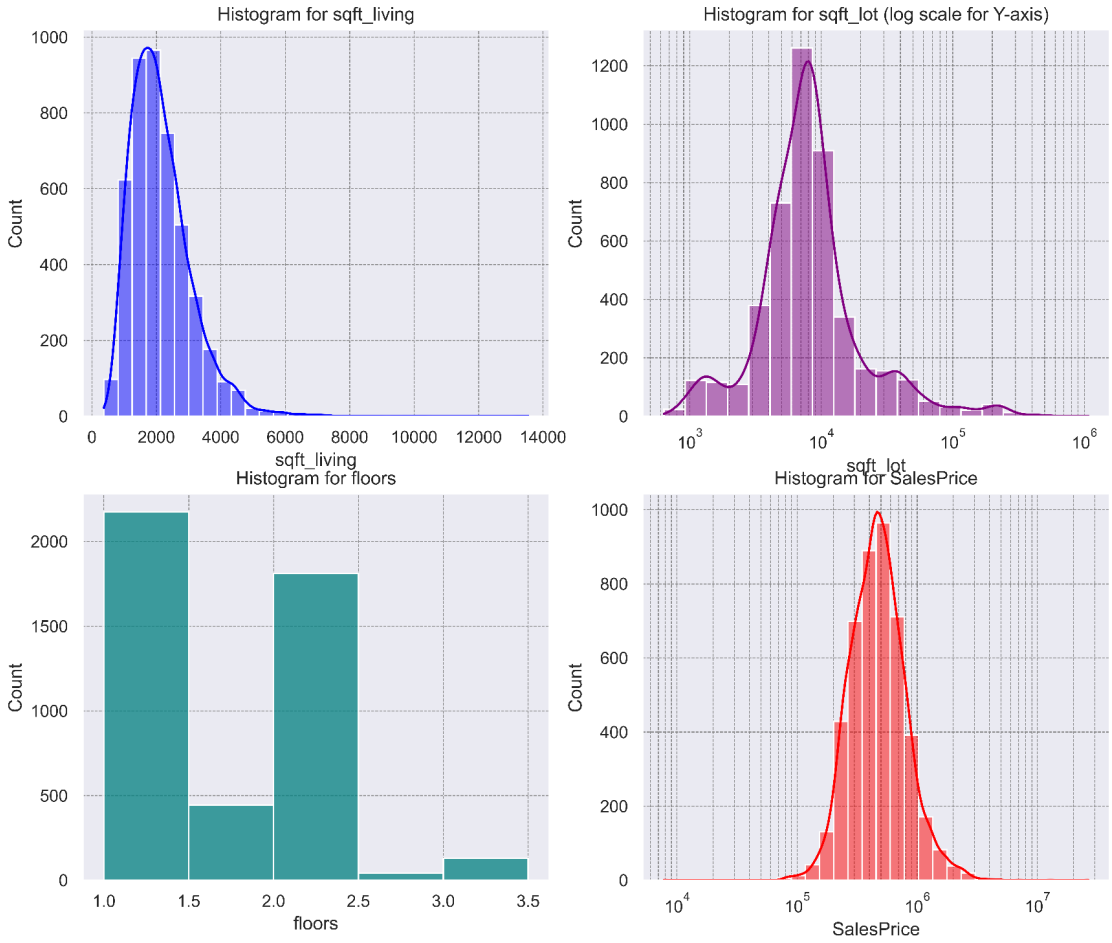
Continuous variables here are:

- sqft_living
- sqft_lot
- sqft_above
- sqft_basement
- SalesPrice

3.2.3. Visualizing the distribution of each feature (sqft_living, sqft_lot, floors, SalesPrice):

We have used histogram plot to visualize the distribution for the features: sqft_living, sqft_lot, floors and SalesPrice.
The histogram plot shows a normal distribution for the features sqft_living, sqft_lot and SalesPrice. As the data for sqft_lot and SalesPrice is highly skewed, a log plot for the corresponding feature makes it easier to understand and visualize these parameters.
The min, max and median values for each of these features is listed below:

Feature	Statistical Value		
	Min	**Median**	**Max**
sqft_living	370	1980	13540
sqft_lot	638	7683	1074218
floors	1	1.5	3.5
Sales Price	7800	465000	26590000



3.2.3. Machine Learning Models:

Linear Regression (Single Variable)

Implementation of linear regression model using the "sqft_lot" feature as the independent variable and "SalePrice" as the target variable along with coef and intercept.

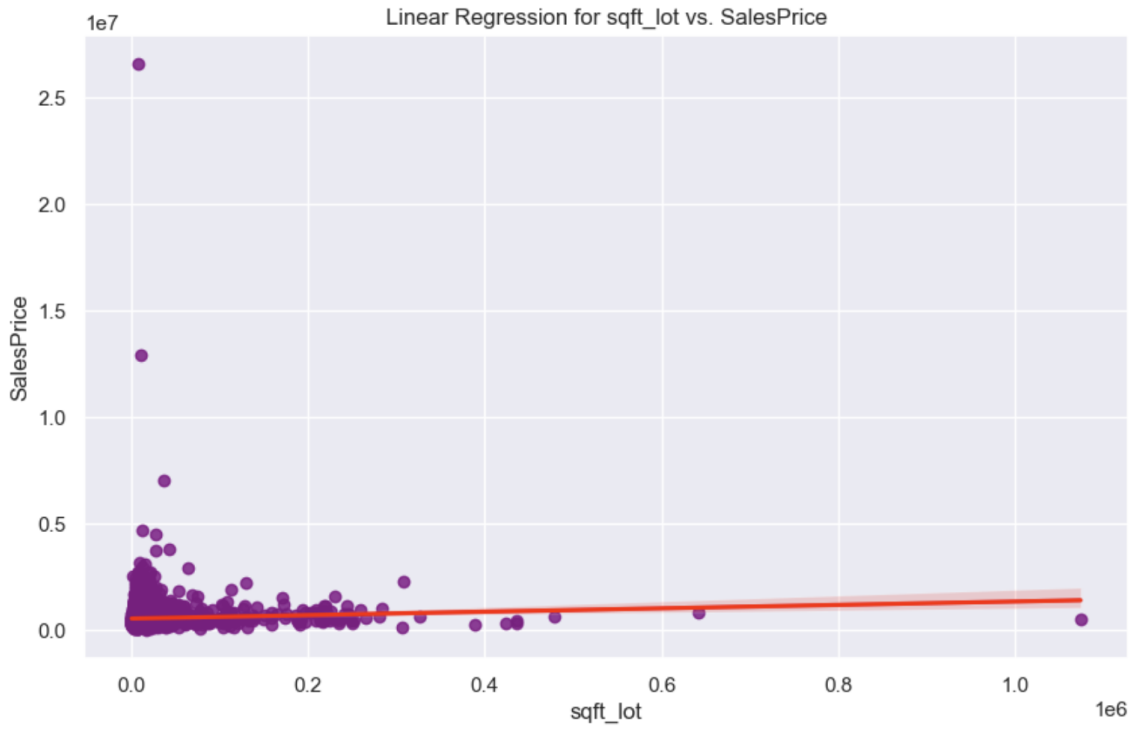
```python
import numpy as np
from sklearn.linear_model import LinearRegression

X = house_price[['sqft_lot']]  # Independent variable
y = house_price['SalesPrice']    # Target variable

# Create a linear regression model
model = LinearRegression()

# Fit the model
model.fit(X, y)

# Print coefficients and intercept
coef = model.coef_[0]
intercept = model.intercept_

print(f'Coefficient: {coef}')
print(f'Intercept: {intercept}')

```

```
Coefficient: 0.7988756407233849
Intercept: 545050.9360372869
```

```python
from sklearn.metrics import mean_squared_error as mse

predictions = model.predict(X)
sse = mse(y, predictions) * len(y)

print(f'Sum of Squared Errors (SSE): {sse}')

r2 = r2_score(y, predictions)
print(f'R-squared (R2): {r2}')
```

```
Sum of Squared Errors (SSE): 1443615835122380.5
R-squared (R2): 0.002611240377665469
```

Plotting the regression line along with the actual data points



LinearRegression function from sklearn.linear_model library and comparison of the coef and intercept:

```
Comparing the coef and intercept of the model


Manual Implementation - Coefficient and Intercepts
 "New_coef": 0.7988756407233877,
 Intercept: 545050.9360372869


Sklearn Linear Regression - Coefficient and Intercepts
 "sklearn_coef": 0.7988756407233849,
 Intercept: 545050.9360372869

 Manual Implementation
  - Sum of Squared Errors (SSE): 1443615835122380.5


 Sklearn LinearRegression
  - Sum of Squared Errors (SSE): 1443615835122380.5
```

Observation:

The obtained results suggest that both the models yield the same statistical output.

Linear Regression (Multivariate):

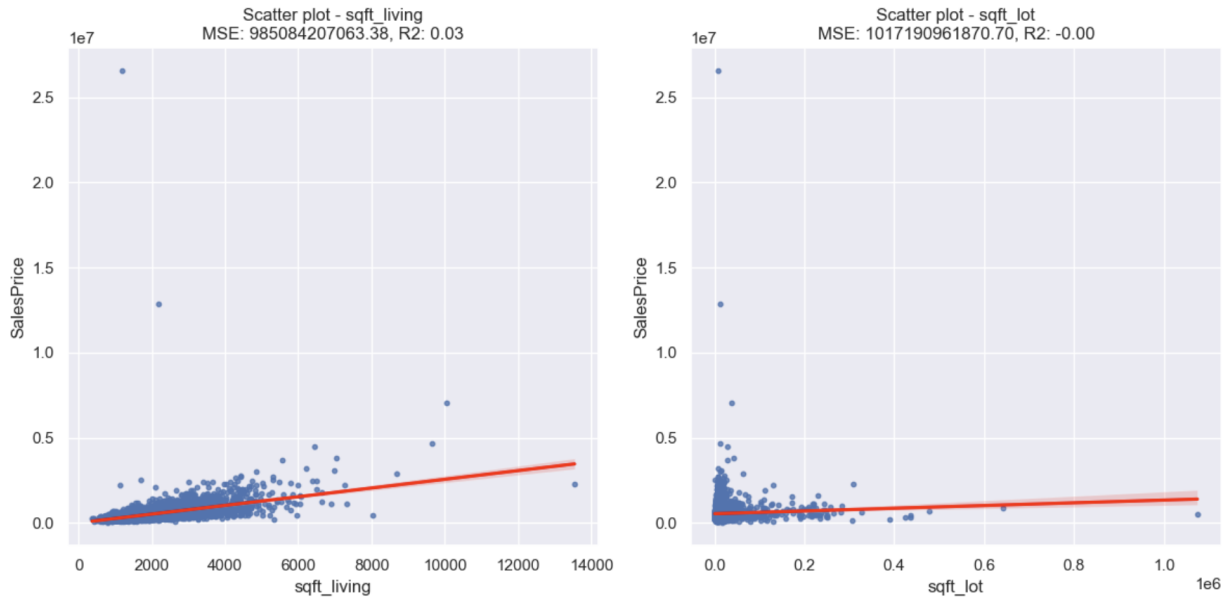
LinearRegression function from sklearn.linear_model library to include multiple features sqft_living, sqft_lot along with the coef and intercept.

```
X = house_price[['sqft_living', 'sqft_lot']]
y = house_price['SalesPrice']


model = LinearRegression()

# Fit the model
model.fit(X, y)

# Print coefficients and intercept
coef = model.coef_
intercept = model.intercept_

print(f'Coefficients: {coef}')
print(f'Intercept: {intercept}')

```

```
Coefficients: [260.69917139  -0.67440183]
Intercept: 9206.834443249507
```

R-squared (R2) score = 0.19408281030398478

Visualizing the relationships between the selected features and SalePrice:



The above image shows model for target variable SalesPrice vs the feature variables sqft_living and sqft_lot:

- The models are inaccurate as the R2 value are very low and very high MSE for both the features.

Polynomial Regression:

Polynomial feature's function to implement a polynomial regression model of degree 2 for the features sqft_lot and the target variable and plots for the polynomial regression curve along with the actual data points.

```
#input
X = house_price[['sqft_lot']]

#output
y = house_price[['SalesPrice']]
```

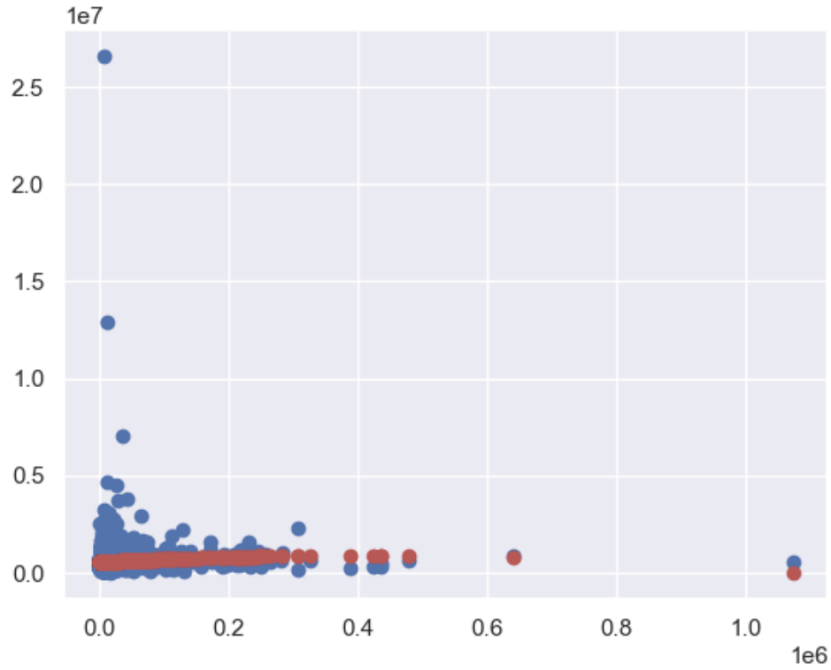
```
# Taking polynomial with degree 2
polynomial_features = PolynomialFeatures(degree = 2, include_bias = False)
Xp = polynomial_features.fit_transform(X)
```

```
pr = LinearRegression()
pr.fit(Xp,y)
```

```
#B1*

pr.coef_

print("B1* : ", pr.coef_)

#B0*

pr.intercept_

print("B0* : ", pr.intercept_)
```

```
B1* :  [[ 1.67763364e+00 -2.01883888e-06]]
B0* :  [535043.59801874]
```

R-squared (R2) score: 0.0046264126724733234



Experimentation with different polynomial degrees to find the best fit:

We will build polynomial model with **degree 10** and check R2 score

```
# Taking polynomial with degree 10
polynomial_features = PolynomialFeatures(degree = 10, include_bias = False)
Xp = polynomial_features.fit_transform(X)
```

```
pr = LinearRegression()
pr.fit(Xp,y)
```

▾ LinearRegression

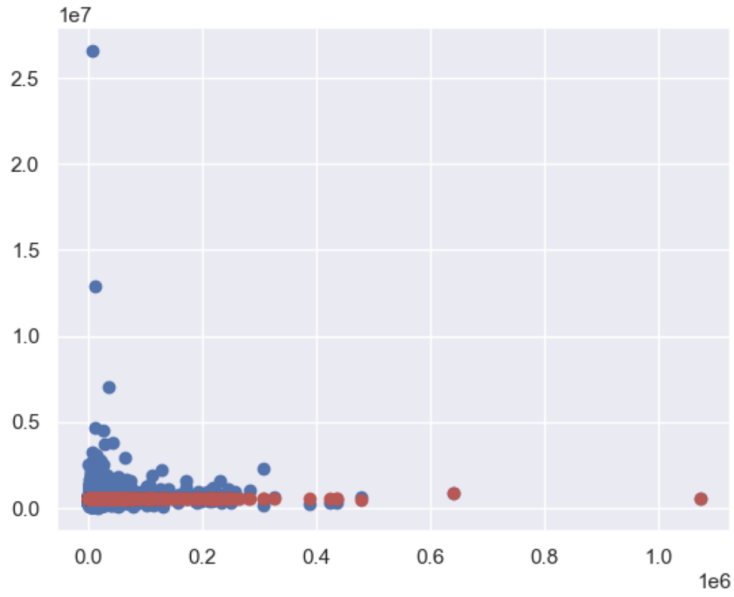
LinearRegression()

```
#B1*

pr.coef_

print("B1* : ", pr.coef_)

#B0*

pr.intercept_

print("B0* : ", pr.intercept_)
```

```
B1* :  [ 9.40838746e-43 -3.12994159e-43  1.88626719e-48  3.67871157e-50
 -2.31785268e-52  1.04989925e-52 -1.29295340e-54  9.36738044e-55
 -4.07831529e-56  0.00000000e+00  3.08936981e-69  1.20551526e-67
  4.18162218e-66  7.39213480e-64 -1.45505092e-66  2.17450983e-65
  9.32107208e-64  2.57917646e-62  1.04023543e-60  2.75104846e-58
 -1.31205615e-62  1.10610278e-61  6.16224720e-60  1.98331391e-58
  4.77446271e-57  2.74194115e-55  9.77832544e-53 -1.40781775e-58
  6.10230652e-59  3.02505246e-56  1.26182302e-54  3.72195693e-53
  7.27297402e-52  7.26904178e-50  3.03277409e-47 -1.62662685e-54
 -9.59310311e-54  5.90223163e-54  4.89242548e-51  2.20330203e-49
  5.85267208e-48  6.11192615e-47  1.49011948e-44  6.51272126e-42
 -1.94144881e-50 -2.02179511e-49 -2.85344558e-48 -2.84054637e-47
  3.33997612e-46  2.82141875e-44  6.01532619e-43 -1.15858874e-41
 -1.45382454e-42  3.21146991e-45 -2.36414753e-46 -3.22184038e-45
 -5.94657128e-44 -1.08167535e-42 -1.68908935e-41 -1.74868793e-40
  8.43950185e-42 -1.61069060e-44 -5.49111438e-46  6.17128965e-48
 -1.22349200e-50]
B0* :  555813.6716552239
```

```
# R2 score
R2_Poly_regression = r2_score(y,pr.predict(Xp))
print("R2 : ",R2_Poly_regression)
```

```
R2 :  6.678120862191328e-05
```



We will build polynomial model with **degree 20** and check R2 score

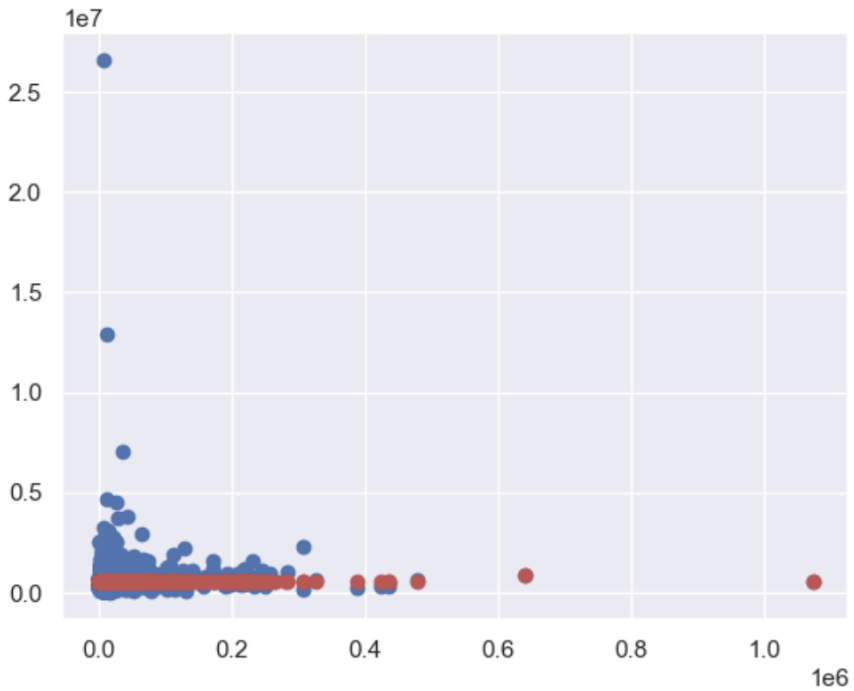
```
# Taking polynomial with degree 20
polynomial_features = PolynomialFeatures(degree = 20, include_bias = False)
Xp = polynomial_features.fit_transform(X)
```

```
pr = LinearRegression()
pr.fit(Xp,y)
```

```
▾ LinearRegression
LinearRegression()
```

```
# R2 score
R2_Poly_regression = r2_score(y,pr.predict(Xp))
print("R2 : ",R2_Poly_regression)
```

```
R2 :  0.0020626470877052 05
```



Observation:

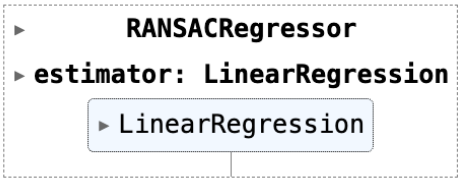
R2 value is least for degree= 10 and highest for degree = 20

Applying RANSAC (Random Sample Consensus) to fit a robust linear regression model to the features sqft_lot and the target variable:

```
from sklearn.linear_model import RANSACRegressor
```

```
#input
X = house_price[['sqft_lot']]

#output
y = house_price[['SalesPrice']]
```

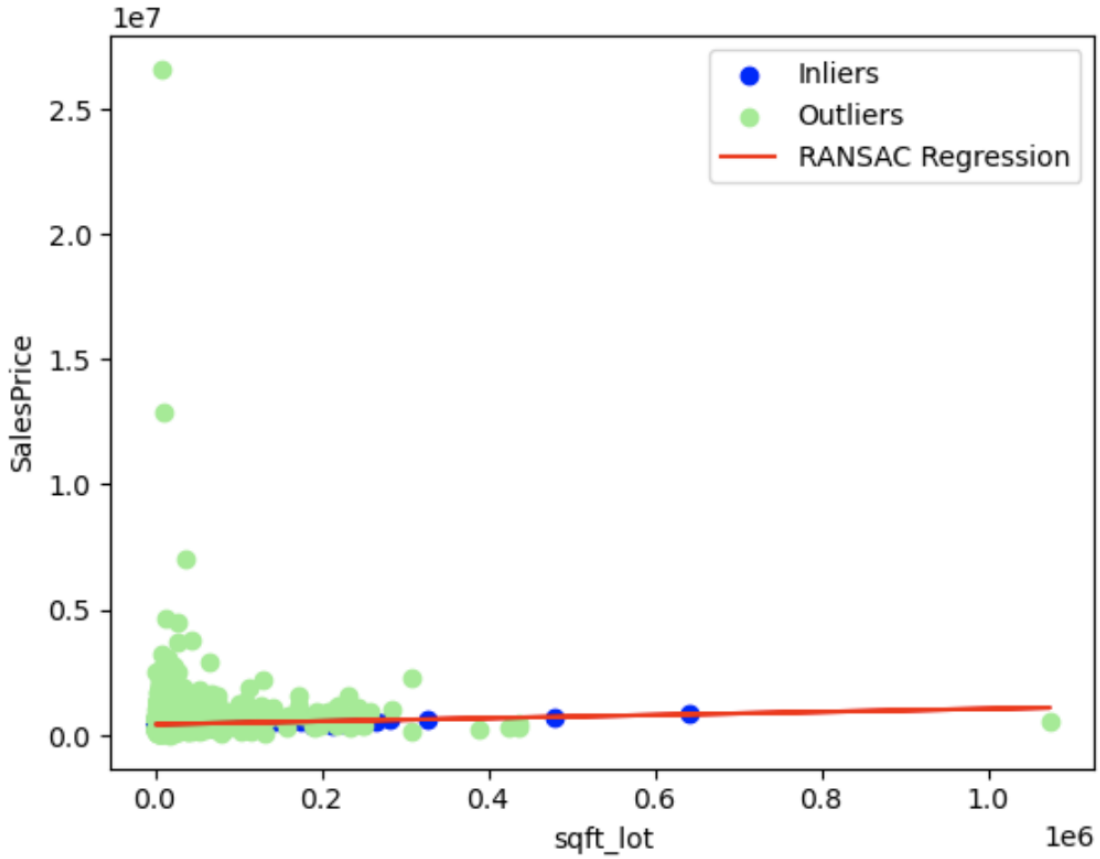
```
ransac = RANSACRegressor(LinearRegression(), max_trials=100, min_samples=50)
ransac.fit(X, y)
```



Printing coef and intercept and Visualizing the plot wrt inliers and outliers with R-squared (R2) score with and without inliers.

```
#B1*
print("B1* : ", ransac.estimator_.coef_)

#B0*
print("B0* : ",ransac.estimator_.intercept_)
```

```
B1* :  [[0.46770074]]
B0* :  [441743.92290971]
```




```
# R-squared score
R2_RANSAC = r2_score(y,ransac.predict(X))
print("R2 : ",R2_RANSAC )
```

```
R2 :  -0.04422492535050959
```

```
# R-squared score with inliers
R2_RANSAC = r2_score(y[inlier_mask],ransac.predict(X[inlier_mask]))
print("R2 with inliers: ",R2_RANSAC )
```

```
R2 with inliers:  0.04610825765025062
```

```
# R-squared score without inliers
R2_RANSAC_without_inlier = r2_score(y[~inlier_mask],ransac.predict(X[~inlier_mask]))
print("R2 with outliers: ",R2_RANSAC_without_inlier )
```

```
R2 with outliers:  -0.1027269286270609
```

Conclusion:

A comparison between the various models using Linear Regression (Single Variable), Linear Regression (Multivariate), Polynomial Regression and RANSAC (Robust Regression) is given below based on their respective $\mathbf{R^2}$ values.Typically we have to choose a high R-squared value which predicts the accuracy of the model. A higher $\mathbf{R^2}$ value indicates a higher accuracy of the model.

Regression Type	**Obtained R^2**
Linear Regression (Single Variable)	0.002 (for feature sqft_lot)
Linear Regression (Multivariate)	0.00 (for feature sqft_lot)
Polynomial Regression	0.002 (Degree=20)
RANSAC (Robust) Regression	0.046 (with inliers)

From the results obtained through various regression models, we can observe that the R2 value is the highest for RANSAC regression suggesting that it provides a better accuracy model when compared to all other regression techniques.

3.3. Part3: Life Expectancy Dataset

3.3.1. Exploration of the dataset:

Displaying the dataset after loading:

```
1 life_exp.head()
```

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	...	Polio	Total expenditure	Diphtheria	HIV/AIDS	
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.279624	65.0	1154	...	6.0	8.16	65.0	0.1	584.259
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.523582	62.0	492	...	58.0	8.18	62.0	0.1	612.696
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.219243	64.0	430	...	62.0	8.13	64.0	0.1	631.744
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.184215	67.0	2787	...	67.0	8.52	67.0	0.1	669.959
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.097109	68.0	3013	...	68.0	7.87	68.0	0.1	63.537

5 rows × 22 columns

Displaying the data types for each column values:

```
Data columns (total 22 columns):
 #   Column                           Non-Null Count  Dtype  
---  ------                           --------------  -----  
 0   Country                          2938 non-null   object 
 1   Year                             2938 non-null   int64  
 2   Status                           2938 non-null   object 
 3   Life expectancy                  2938 non-null   float64
 4   Adult Mortality                  2938 non-null   float64
 5   infant deaths                    2938 non-null   int64  
 6   Alcohol                          2938 non-null   float64
 7   percentage expenditure           2938 non-null   float64
 8   Hepatitis B                      2938 non-null   float64
 9   Measles                          2938 non-null   int64  
 10  BMI                              2938 non-null   float64
 11  under-five deaths                2938 non-null   int64  
 12  Polio                            2938 non-null   float64
 13  Total expenditure                2938 non-null   float64
 14  Diphtheria                       2938 non-null   float64
 15   HIV/AIDS                        2938 non-null   float64
 16  GDP                              2938 non-null   float64
 17  Population                       2938 non-null   float64
 18  thinness  1-19 years             2938 non-null   float64
 19  thinness 5-9 years               2938 non-null   float64
 20  Income composition of resources  2938 non-null   float64
 21  Schooling                        2938 non-null   float64
dtypes: float64(16), int64(4), object(2)
memory usage: 505.1+ KB
```

Summary Statistics:

```
1 life_exp.describe()
```

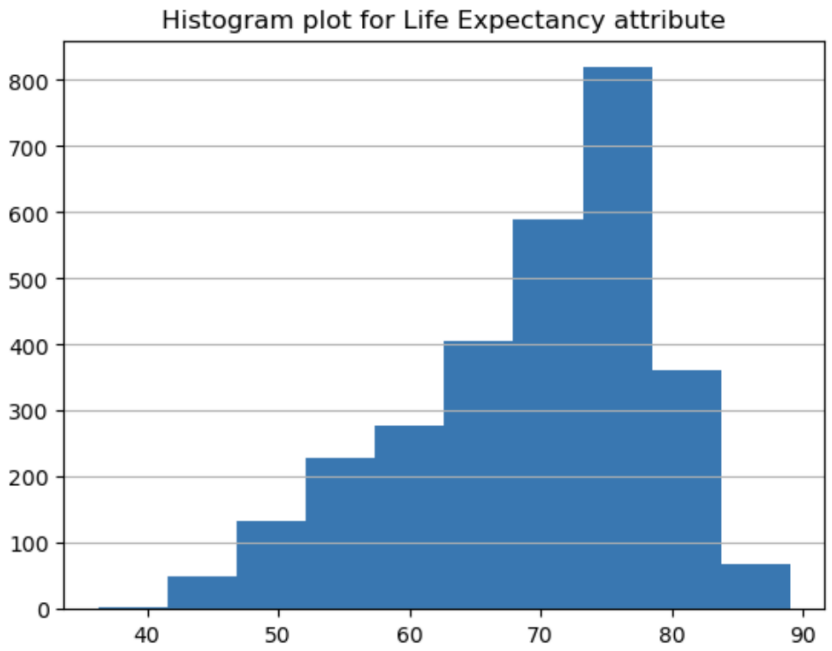
	Year	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	BMI	under-five deaths	Polio
count	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000	2938.000000
mean	2007.518720	69.234717	164.725664	30.303948	4.546875	738.251295	83.022124	2419.592240	38.381178	42.035739	82.617767
std	4.613841	9.509115	124.086215	117.926501	3.921946	1987.914858	22.996984	11467.272489	19.935375	160.445548	23.367166
min	2000.000000	36.300000	1.000000	0.000000	0.010000	0.000000	1.000000	0.000000	1.000000	0.000000	3.000000
25%	2004.000000	63.200000	74.000000	0.000000	1.092500	4.685343	82.000000	0.000000	19.400000	0.000000	78.000000
50%	2008.000000	72.100000	144.000000	3.000000	3.755000	64.912906	92.000000	17.000000	43.500000	4.000000	93.000000
75%	2012.000000	75.600000	227.000000	22.000000	7.390000	441.534144	96.000000	360.250000	56.100000	28.000000	97.000000
max	2015.000000	89.000000	723.000000	1800.000000	17.870000	19479.911610	99.000000	212183.000000	87.300000	2500.000000	99.000000

Identify and specify the target variable from the dataset:

- The Life Expectancy dataset has 22 fields and 2938 records from 193 countries.
- The dataset is a collection of data about various factors like- 'Adult Mortality','infant deaths', 'Alcohol', 'percentage expenditure', 'Hepatitis B','Measles', 'BMI', 'under-five deaths ', 'Polio', 'Total expenditure' etc collected over few years from different countries.
- These attributes might contribute towards determining the 'Life expectancy' index of a country. So, the target variable here is 'Life expectancy' attribute.

Visualizing the Target Variable: LifeExpectancy

Histogram:



Observing Negative skewness.

BoxPlot:

- The values of Life Expectancy attributes ranges from 36.3 to 89 with standard deviation of 9.5 approx.
- The bosplot of this attribute shows that there are some outliers below Q1(25%)-1.5IQR point in the graph and it is responsible for bringing down the mean value from the median point of this attribute value.
- Also, the histogram plot is telling that there exist highest number of data points with life expectancy value between the range of 70 to 80.

Categorizing the columns into categorical and continuous.
The columns are categorized into categorical and continuous variables as follows:

```
Country                           object
Year                               int64
Status                            object
Life expectancy                  float64
Adult Mortality                  float64
infant deaths                      int64
Alcohol                          float64
percentage expenditure           float64
Hepatitis B                      float64
Measles                            int64
BMI                              float64
under-five deaths                  int64
Polio                            float64
Total expenditure                float64
Diphtheria                       float64
 HIV/AIDS                        float64
GDP                              float64
Population                       float64
thinness  1-19 years             float64
thinness 5-9 years               float64
Income composition of resources  float64
Schooling                        float64
dtype: object
```

So, among 22 columns in Life expectancy dataset, Country and Status are categorical variables and rest of the columns are continous variables.

Categorical variables in the dataset

```
1 cat_var_col_array=cat_var.columns.tolist()
2 cat_var_col_array
```

```
['Country', 'Status']
```

Continuous variables in the dataset

```
cont_var_col_array=life_exp.loc[:, ~life_exp.columns.isin(cat_var_col_array)].columns.tolist()
cont_var_col_array
```

```
['Year',
 'Life expectancy',
 'Adult Mortality',
 'infant deaths',
 'Alcohol',
 'percentage expenditure',
 'Hepatitis B',
 'Measles',
 'BMI',
 'under-five deaths ',
 'Polio',
 'Total expenditure',
 'Diphtheria',
 ' HIV/AIDS',
 'GDP',
 'Population',
 'thinness  1-19 years',
 'thinness 5-9 years',
 'Income composition of resources',
 'Schooling']
```

Identify the unique values from each column:

Checking the categorical variable - Status

```
life_exp.Status.unique()
```

```
array(['Developing', 'Developed'], dtype=object)
```

Observation:

- Status is categorical variable and has 2 unique values
  - Developing
  - Developed

```
print( 'We have data from %d number of countries' % len(life_exp.Country.unique()))
```

```
We have data from 193 number of countries
```

- Country is another categorical variable in the dataset with 193 unique values. So, we have data from 193 countries.

Identify the Missing values and compute the missing values with mean, median or mode based on their categories. Also explain why and how you performed each imputation:

Checking the nulls:

```
1 life_exp.isnull().sum()
```

```
Country                          0
Year                             0
Status                           0
Life expectancy                  0
Adult Mortality                  0
infant deaths                    0
Alcohol                          0
percentage expenditure           0
Hepatitis B                      0
Measles                          0
BMI                              0
under-five deaths                0
Polio                            0
Total expenditure                0
Diphtheria                       0
 HIV/AIDS                        0
GDP                              0
Population                       0
thinness  1-19 years             0
thinness 5-9 years               0
Income composition of resources  0
Schooling                        0
dtype: int64
```

Observation:

- life expectancy dataset does not have any null values in any columns.
- So, we will check the dataset for the presensence of zero values in any of these columns

Checking zero values in each column except target variable

- Following attributes are having zero value fields:
    - infant_deaths_zero_cnt,
    - percentage_expenditure_zero_cnt,
    - Measles_zero_cnt,
    - under_five_deaths_zero_cnt,
    - Income_composition_of_resources_zero_cnt,
    - Schooling_zero_cnt fields
- We will replace all these zero values with null values.

Check for the outliers in each column using the IQR method:

Checking null values in each columns

```
1 life_exp.isnull().sum()
```

```
Country                             0
Year                                0
Status                              0
Life expectancy                     0
Adult Mortality                     0
infant deaths                     848
Alcohol                             0
percentage expenditure            611
Hepatitis B                         0
Measles                           983
BMI                                 0
under-five deaths                 785
Polio                               0
Total expenditure                   0
Diphtheria                          0
 HIV/AIDS                           0
GDP                                 0
Population                          0
thinness  1-19 years                0
thinness 5-9 years                  0
Income composition of resources   130
Schooling                          28
dtype: int64
```

Observation:

- infant deaths and under-five deaths field values are having zero for multiple developed and deveoping countries for consecutive years which is not quite convinvingly correct. So these fields need value imputation as the correct values are missing here.
- percentage expenditure field is zero for few countries even though they have total expenditure as not null. So this field needs imputation.
- Since all of these fields are continuous fields, we will be imputing all these null values with the corresponding median values of the attribute. Thus, distribution of these continuous attribute will remain unchanged.

Checking null values in each columns after median value imputation

```
: 1 life_exp.isnull().sum()
```

```
: Country                          0
  Year                             0
  Status                           0
  Life expectancy                  0
  Adult Mortality                  0
  infant deaths                    0
  Alcohol                          0
  percentage expenditure           0
  Hepatitis B                      0
  Measles                          0
  BMI                              0
  under-five deaths                0
  Polio                            0
  Total expenditure                0
  Diphtheria                       0
   HIV/AIDS                        0
  GDP                              0
  Population                       0
  thinness  1-19 years             0
  thinness 5-9 years               0
  Income composition of resources  0
  Schooling                        0
  dtype: int64
```

Observation for Check for the outliers in each column using the IQR method

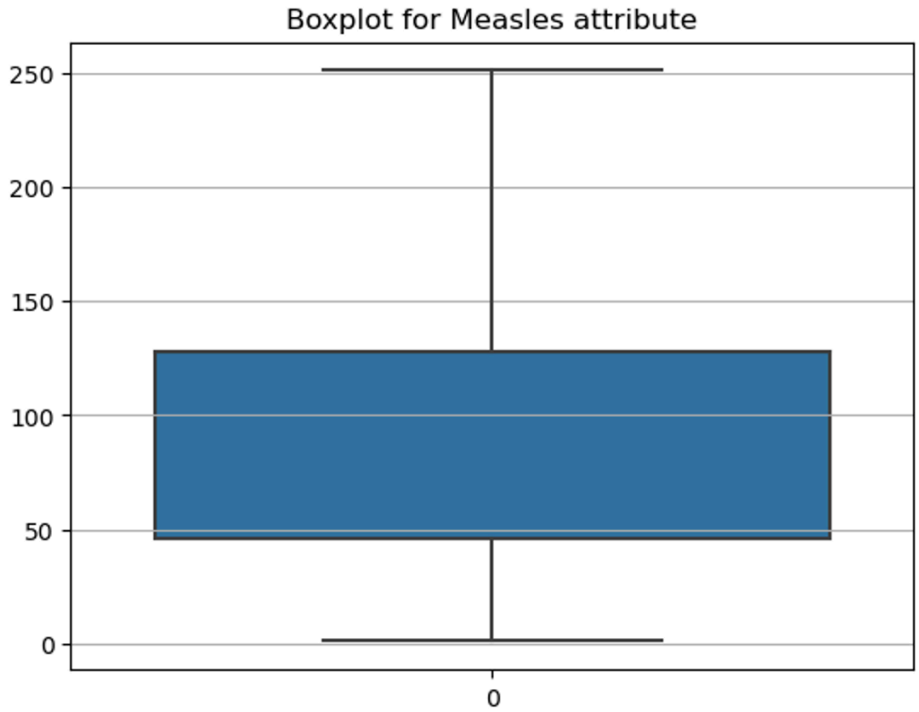
- A data point is called outlier when it is outside the range of 1.5 times the IQR below the 25th percentile or above the 75th percentile.
- Here, we will calculate the 25th and 75th percentiles of the dataset and then calculate the IQR to identify outliers.
- We will find out outlier data from all the cotinuous variable fields

Displaying all the continuous variable:

```
['Year',
 'Life expectancy',
 'Adult Mortality',
 'infant deaths',
 'Alcohol',
 'percentage expenditure',
 'Hepatitis B',
 'Measles',
 'BMI',
 'under-five deaths ',
 'Polio',
 'Total expenditure',
 'Diphtheria',
 ' HIV/AIDS',
 'GDP',
 'Population',
 'thinness  1-19 years',
 'thinness 5-9 years',
 'Income composition of resources',
 'Schooling']
```

Impute the outliers and impute the outlier values with mean, median or mode based on their categories:

Here we have imputed outliers with median values and plotted the attribute.



3.3.2. Machine learningModels:

Identifying and performing label encoding on certain columns

(a) Specify and explain on which columns you perform and why.

Columns Selection for Label encoding and Explanation:

Country and Status columns in this dataset have categorical values and represented in String format- name of the countries. We can perform label encoding for Country and Status columns. In this way Country, Status columns will be converted into numerical values and can contribute towards predictive algorithms.

(b) Explain what is label encoding and how it changes the dataset.

Label Encoding and how it changes the dataset:

Label Encoding is a way to handle and encode categorical values. By Label encoding text categorical field were ranked based on alphabetical value. Here Country and Status columns were initially represeted as categorical value and as a text field. After label encoding Countries were represented as 0,1,2.. values based on their alphabetic rankings.

Perform data normalization on ‘Adult Mortality', 'BMI', 'GDP’ numerical columns using StandardScaler()

Data normalization was performed on only ‘Adult Mortality', 'BMI', 'GDP’ columns using StandardScaler() method. After normalization all these attribute values came within similar range. Here is snippet of standardized values.

```
#Let check after performing transformation
scaled_data
```

```
array([[ 1.06432757, -0.96734874, -0.73578463],
       [ 1.14159693, -0.99243406, -0.72133961],
       [ 1.11262092, -1.01751937, -0.71166375],
       ...,
       [-0.77081972, -0.60612024, -1.00343439],
       [-0.08505416, -0.62618849, -0.75390453],
       [-0.08505416, -0.64625674, -0.75452853]])
```

3.11. Compute a correlation matrix and plot the correlation using a heat map and answer the following questions:

The Features which are Most Positively Correlated with target variable:

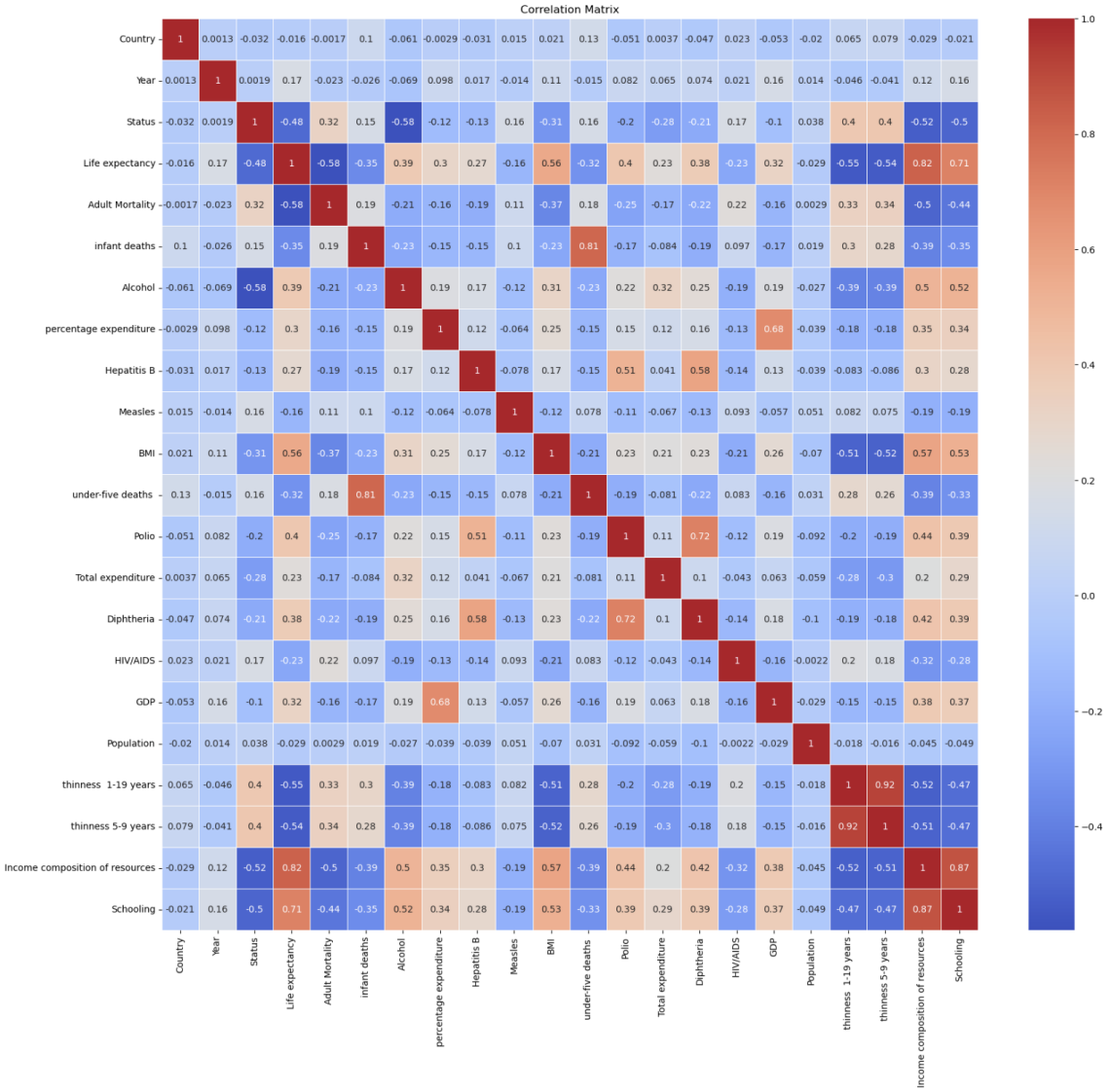
Observation:

- Life Expectancy is the target variable for this dataset. Based correlation heatmap matrix, following variable are highly positively correlated to 'Life Expectancy' variable:
    - a. Schooling - correlation value is 0.71
    - b. Income compositions of resources - correlation values is 0.82
    - c. BMI - correlation value is 0.56

The Features which are Most Negatively Correlated with target variable

Observation:

- Based correlation heatmap matrix, following variable are highly negatively correlated to 'Life Expectancy' variable:
    - a. Adult mortality - correlation value is -0.58
    - b. thinness 1-19 years - correlation value is -0.55
    - c. thinness 5-9 years - correlation value is -0.54
    - d. Status - correlation value is -0.48



Drop the column 'country' from the dataset and split the dataset into training and testing in a 70:30 split:

We will check shape of each test and train data after splitting in into test and train

```
1 x_train.shape
```

```
(2056, 20)
```

```
1 y_train.shape
```

```
(2056, 1)
```

```
1 x_test.shape
```

```
(882, 20)
```

```
1 y_test.shape
```

```
(882, 1)
```

Linear regression model using the training and testing datasets and compute mean absolute error.

Observation:

- We will perform scaler transformation on train and test data and perform linear regression from scikit learn library.
- Then we will fit linear regression on train data and perform prediction on test dataset.
- And at the end we check regression score and mean absolute error

```
1 ### Regression score
2
3 linear_regressor.score(x_test, y_test)
4
```

```
0.7587405401278664
```

```
1 from sklearn.metrics import mean_absolute_error,mean_squared_error
2
3 mae = mean_absolute_error(y_true=y_test,y_pred=Y_pred)
4
5 print("Mean Absolute Error :",mae)
6
7 mse= mean_squared_error(y_true=y_test,y_pred=Y_pred)
8 print("Mean Squared Error :",mse)
```

```
Mean Absolute Error : 3.3088645185294734
Mean Squared Error : 22.66302496970964
```

Mean Absolute Error is around 3.3 meaning is that there is around 3.3 values difference between the actual and predicted values on average. Also, linear regression gave mean squared error as 22.66

Linear regression model using mini batch gradient descent and stochastic gradient descent with alpha=0.0001, learning rate='invscaling', maximum iterations =1000, batch size=32 and compute mean absolute error:

To perform gradient descent on the given dataset, we will perform scaler transformation on both test and train data and pass them to Gradient Descent Stochastic function.

```
# Making prediction
y_sgd_pred = stochastic_reg.predict(x_sgd_train)

print("Train: Mean Absolute Error: %.2f"
      % mean_absolute_error(y_train, y_sgd_pred))
```

```
Train: Mean Absolute Error: 3.40
```

```
print("Train: Mean Squared Error: %.2f"
      %mean_squared_error(y_train, y_sgd_pred, squared=False))
```

```
Train: Mean Squared Error: 4.84
```

So, using Stochastic Gradient Descent on linear regression, we got mean absolute error as 3.40 and Mean squared error as 4.84 . While Mean absolute error got increased a little from normal linear regression, mean squared error got improved from linear regression method.

Linear regression model using mini batch gradient descent with learning rate = 0.0001, maximum iterations =1000 and batch size=32. Manually without using any scikit learn libraries:

```
a_optimal,b_optimal = mini_batch_gradient_descent(x, y, slope_initial, intercept_initial, learning_rate,batch_si

print(f"Optimal optimal slope: {a_optimal}, Optimal bias: {b_optimal}")
```

```
Epoch 983/1000, SSE: 425.4973605581582
Epoch 984/1000, SSE: 1420.4548964498201
Epoch 985/1000, SSE: 1277.2920621353876
Epoch 986/1000, SSE: 849.8294213879201
Epoch 987/1000, SSE: 924.8918705643698
Epoch 988/1000, SSE: 886.0834244977619
Epoch 989/1000, SSE: 1015.4430585413595
Epoch 990/1000, SSE: 360.81463773409627
Epoch 991/1000, SSE: 1105.2182480873162
Epoch 992/1000, SSE: 431.9644056342674
Epoch 993/1000, SSE: 398.0037937898739
Epoch 994/1000, SSE: 941.3844277543637
Epoch 995/1000, SSE: 518.2515969674888
Epoch 996/1000, SSE: 830.35539981223
Epoch 997/1000, SSE: 1034.9806483097636
Epoch 998/1000, SSE: 807.824004391175
Epoch 999/1000, SSE: 622.8905094250294
Epoch 1000/1000, SSE: 677.0745278218722
Optimal optimal slope: -0.1450608664640885, Optimal bias: 69.2230977752772
```

Observation:

- Here we gave created user defined function - mini_batch_gradient_descent to compute mini batch Gradient Descent on linear regression using batch size 32 and used 1000 epochs.
- For each iterations we calculated Sum of Squared Error(SSE) and found that it keeps going zigzag on each iterations.
- Lowest SSE we found at Epoch 541/1000 as 142.3638813218416 whereas at the end of 1000 epoch we found SSE approximately as 441.23633 along with optimal slope and bias as -0.14421800840034235 and 69.2243572826775.
- Also, with each run it keeps on changing with each run as we take random data with batch size 32.

Comparison of the results from each approach and also explanation of the difference between mini batch gradient descent and stochastic gradient descent.

- In Stochastic Gradient Descent using Sklearn library, mean squared error found is 4.84.
- In minibatch without sklearn the sum of squared error keeps fluctuating in order to find global minima and at epoch 1000 it is around 645.65 and keeps on changing due to random selection of data points in each run .
- So clearly using mini batch with size batch size 32 , mean squared error = (1/batch size) * sum of squared error = (1/32) * 645.65 ~ 20.17 which is still lesser than what we found from Stochastic Gradient Descent.

However, mini batch gives better result than linear regression method.So we can conclude that Stochastic Gradient descent approach is best tahn mini batch and normal liear regression methods.

Conclusion

Life expectancy of a person is determined by many factors including their Country's GDP, status to their Country's overall health wise behavior. Our analysis shows that Life expectancy of a country might get higher with average BMI of a person in that country along with how the people is getting schooled and income composition. On the contrary, high adult mortality , thinness in a person under 17 years old might lower life expectancy. We have performed simple linear regression, linear regression with stochastic gradient descent and mini batch gradient descent to predict life expectancy based on the given attributes. With Stochastic gradient descent we are able to predict life expectancy with MSE at 4.84 which is better prediction model than the other two.